tmux documentation

§07 — Anti-Bluff Test Strategy for Termux

Revision: 1 **Last modified:** 2026-05-22T07:20:02Z **Authority:** vasic-digital tmux project (research-only) **Maintainer:** milosvasic **Scope:** How to run the §103 four-layer + §11.4 anti-bluff covenant on Android via Termux

---

1. The challenge

Our anti-bluff posture requires positive captured runtime evidence per §11.4.5 — not just exit codes. On Linux/macOS we have:

- Direct shell access on the SUT.
- `commit_all.sh` → push → CI → tested on Linux.
- Nezha-Linux remote test bridge for the existing v1.0.9 spec.

For Termux we need an equivalent capture path that does NOT require:

- `adb shell run-as com.termux` — this DOES NOT WORK. Termux's `AndroidManifest.xml` does NOT declare `android:debuggable="true"` (security posture), so `run-as` refuses access. Confirmed in numerous Termux GitHub discussions.
- Rooting the test device.
- Installing a custom Termux build.

2. The SSH-driven harness — RECOMMENDED

The cleanest approach is to use **Termux's own sshd as the test harness's remote-host channel**. The host machine (Linux or macOS dev box) ssh's into the phone on port 8022 and drives tests over SSH. This is identical in shape to our existing nezha remote-test pattern.

2.1 Setup contract

The test device (a phone or tablet running Termux) ships with:

- Termux installed from F-Droid.
- OpenSSH (`pkg install openssh`).
- `~/.ssh/authorized_keys` populated with the host machine's CI public key (NO `command=` restriction — we want unrestricted shell for testing).
- `sshd` running on 8022 (started via Termux:Boot autostart script).
- `termux-wake-lock` held permanently.
- Battery optimization disabled.
- Phantom Process Killer disabled.
- Our project cloned at `~/tmux-project`.
- Network access (Wi-Fi LAN or Tailscale).

2.2 Host-side test driver

```
# scripts/test_termux.sh — host-side driver
set -euo pipefail
TERMUX_HOST="${TERMUX_TEST_HOST:-}"
[ -z "$TERMUX_HOST" ] && { echo "SKIP: TERMUX_TEST_HOST not
        set"; exit 0; }

ssh -p 8022 "$TERMUX_HOST"
        'cd ~/tmux-project && git pull && bash scripts/tests/
        run_all.sh'
```

Test scripts run unchanged on the remote because Termux's bash is bash 5.x and our scripts use POSIX features (per §11.4.67). Output is captured back to host, evidence files are pulled via `scp`.

2.3 Captured-evidence per §11.4.5

Each test still must produce positive evidence — same shape as Linux/macOS, just collected over SSH:

- `tmux capture-pane -p` output saved to `~/.tmx/evidence/<test>/<iter>/pane.txt`, then `scp`'d to host.
- For tests asserting RLIMIT_AS enforcement: capture `/proc/$PID/status` lines `VmPeak:`, `VmSize:`, before and after attempting to exceed.
- For tests asserting `RLIMIT_CPU` enforcement: capture the `SIGXCPU` followed by `SIGKILL` via a small trap script.
- For tests asserting OOM survival: capture `/proc/$PID/oom_score_adj` value + tmux-server PID before and after a synthetic memory-pressure event.
- For tests asserting wake-lock effectiveness: capture `dumpsys power | grep -A2 'PARTIAL_WAKE_LOCK'` output BEFORE entering Doze + AFTER simulating Doze.

All evidence files travel back to host and feed the standard `qa-results/` analyser.

3. §11.4.81 four-platform parity table (proposed)

Test	Linux (systemd)	Linux (no-systemd, e.g. Alpine)	macOS	Termux
Memory cap enforced	cgroup `memory.max`	rlimit `RLIMIT_AS`	macOS gap (no enforcement) — SKIP `feature_unsupported: xnu-rlimit-as-unenforced`	rlimit `RLIMIT_AS` (kernel-enforced)
CPU cap enforced	cgroup `cpu.max`	rlimit `RLIMIT_CPU`	rlimit `RLIMIT_CPU`	rlimit `RLIMIT_CPU`
Task cap enforced (per session)	cgroup `pids.max`	NO per-session — `RLIMIT_NPROC` is per-UID — SKIP partial	NO per-session (same) — SKIP partial	NO per-session (same) — SKIP partial; Android 12+ phantom-killer adds an

Test	Linux (systemd)	Linux (no-systemd, e.g. Alpine)	macOS	Termux
				UNRELATED 32-cap
OOM kill priority	`oom_score_adj` + cgroup OOM	`oom_score_adj`	macOS has no OOM killer — SKIP `feature_unsupported: xnu-no-oom-killer`	`oom_score_adj` writable but lmkd may override — degraded
Background survival	systemd-managed	service-manager dependent	login keeps process alive	wake-lock + battery-opt required — degraded
SSH dispatch (`command=`)	works	works	works	works (port 8022)
State daemon flock	works	works	works	works
Shell-init non-TTY skip	works	works	works	works

Each "works" needs CAPTURED-RUNTIME-EVIDENCE per §11.4.5, not just an exit code. The matrix becomes one of the layer-3 runtime tests in the existing `scripts/tests/` directory.

4. Test 31 — `macos_linux_parity.sh` extension to Termux

Spec §7.3 test 31 today reads `case "$(uname -s)" in Darwin) ... ;; Linux) ... ;; esac`. For Termux this becomes:

```
case "$(uname -s)" in
    Darwin) ... ;;
    Linux)
        if [ -n "${TERMUX_VERSION:-}" ]; then
            # Termux branch — same rlimit assertions as macOS
        PLUS
            # the additional RLIMIT_AS check that macOS skips.
            _assert_rlimit_as_enforced
            _assert_rlimit_cpu_enforced
            _assert_rlimit_nproc_enforced
        elif _has_systemd_user; then
            # Real Linux with systemd — cgroup assertions.
            _assert_cgroup_memory_max
            _assert_cgroup_cpu_quota
            _assert_cgroup_tasks_max
        else
            # Linux without systemd — same rlimit assertions as
        macOS/Termux.
            _assert_rlimit_cpu_enforced
```

```
            _assert_rlimit_nproc_enforced
            # RLIMIT_AS works here too but is currently a Linux-
        Alpine-only path.
        fi
        ;;
esac
```

§11.4.4 (test-interrupt-on-discovery) applies: discovering Termux fails the AS check would STOP the cycle until fixed.

5. CI integration

Three deployment options for CI:

5.1 Self-hosted phone in a drawer

- Real phone, plugged in, behind Tailscale, reachable from CI runner.
- Every push triggers `scripts/test_termux.sh`.
- Pros: real device, real Android, real lmkd behaviour.
- Cons: physical maintenance, network reliability, OS update churn.

5.2 Android emulator in CI

- `avdmanager create avd ...` + `emulator -no-window`.
- Termux can be installed in an emulator via `adb install` of the F-Droid APK.
- Pros: reproducible, no physical hardware.
- Cons: emulators do NOT replicate lmkd / Doze / Phantom Process Killer accurately. Most of the WILL-DEGRADE items in §06 can't be tested.
- Verdict: useful for happy-path build smoke; insufficient for full anti-bluff.

5.3 Hybrid: emulator for cheap smoke + scheduled phone-runs for full sweep

- PRs go through emulator (5-min smoke).
- Tagged releases run full sweep on a real phone, blocking the tag until green.
- Mirrors our existing Linux/macOS split (CI does Linux fast, nezha bridge does deep tests).
- Recommended.

6. HelixQA Challenges on Termux

Existing Challenge bank lives at `scripts/challenges/tmux.yaml`. Each Challenge invokes a fixed shell command and asserts on captured output. They run on whatever the SSH-driven test harness lands on, so:

- `tmx_session_resume_cwd` works on Termux unchanged.
- `tmx_ssh_dispatch_nezha` becomes `tmx_ssh_dispatch_termux` with the `Port 8022` host alias.
- `tmx_non_tty_safety` works unchanged.
- `tmx_docs_user_guides_render` works unchanged (pandoc + weasyprint available in Termux via `pkg install pandoc weasyprint`).

A new Challenge `tmx_termux_wake_lock_held` validates the operator-side wake-lock notice mechanism: spawn `tmx new -s foo`, capture stderr, assert it mentions `termux-wake-lock` if `dumpsys power` shows no held lock. (Operator-facing notice — not a hard fail, just a captured warning.)

7. Anti-bluff failure modes specific to Termux

The §11.4.69 sink-side evidence taxonomy applies. Termux-specific risks:

- **Phantom Process Killer firing mid-test.** A passing test gets `SIGKILL`'d at iteration 7 of 10 because Android decided the parent went background. The test framework needs to detect this (signal 9 != normal exit) and re-classify the run as `topology_unsupported: termux-phantom-killer-fired` rather than letting it count as a flake.
- **Doze freeze mid-test.** Same shape — screen lock during a long test → CPU frozen → test wall-clock blows past timeout. Detect via `dumpsys deviceidle | grep STATE` poll; classify as topology issue.
- **lmkd `SIGKILL` during memory-pressure test.** A memory-cap test that asserts the kernel OOM-killed the offending child may instead see lmkd kill the entire Termux process before the kernel OOM fires. Detect by reading `/dev/kmsg` (where lmkd logs its kills) vs `dmesg` (where kernel OOM logs).

Each requires per-Termux test logic. Documented in §06 already; here we map them onto the test-harness side.

8. §11.4.50 deterministic-consistency on Termux

The 3-iteration (or 10-iteration) loop applies. On Termux specifically:

- Iteration latency is HIGHER (slower CPU, slower I/O on flash storage) — overall sweep walltime increases by ~3× vs a Linux SSD.
- Some iterations may fall victim to background-kill mid-iteration. Classification must distinguish "feature is non-deterministic" (FAIL) from "Android killed the test process" (SKIP-with-reason).

Recommend Termux-specific N=3 for fast tests, N=10 reserved for the cycle-validation suite, with explicit retry-on-SIGKILL semantics.

9. Pre-build gate `CM-CROSS-PLATFORM-PARITY` extension

§11.4.81 already defines this gate. When Termux lands, the gate's `case "$(uname -s)"` scanner detects three branches needed:

- `Darwin)`
- `Linux)` + nested `if [ -n "${TERMUX_VERSION:-}" ]`
- `Linux)` + nested `else` (the systemd-or-not split)

The mutation §1.1 pair strips the Termux nested branch → gate FAILs. Standard four-layer integration.

10. Topology detection in tests

The canonical "am I on Termux?" detector is:

```
_is_termux() { [ -n "${TERMUX_VERSION:-}" ] || [ -d /data/data/
        com.termux/files/usr ]; }
```

Belt-and-suspenders — `$TERMUX_VERSION` is the env var, the directory check is the fallback when the env wasn't sourced.

11. Captured-evidence directory layout

```
qa-results/
└── termux/
    ├── 2026-05-22T07-20-02Z/
    │   ├── device.txt           # uname -a, getprop ro.product.model, Ter
    │   ├── 18_state_persistence/
    │   │   ├── iter-1/pane.txt
    │   │   ├── iter-1/state.json
    │   │   ├── iter-2/...
    │   ├── 22_ssh_dispatch_local/
    │   ├── 31_macos_linux_parity/
    │   └── meta-test/
    │       └── mutation-M20/...
```

Symmetric with the existing Linux/macOS layouts.

12. Summary

The test strategy reduces to: **SSH-into-the-phone as we today SSH-into-nezha**. The technical primitives (SSH, capture-pane, evidence files, flock state) all port. The COMPLICATION is that Android's mid-test killers (lmkd, Doze, Phantom Process Killer) introduce a new failure class that must be detected and classified as topology issues, not test flakes. Our existing §103 + §11.4 machinery accommodates this via the SKIP-with-reason vocabulary.

Effort estimate for test-harness adaptation: ~200 LOC of new bash (`scripts/test_termux.sh` + the per-test Termux branches in tests 18-31 + the new `_is_termux` helper in the anti-bluff lib). Plus the operator-side phone-prep checklist (§05).

Sources

- https://source.android.com/docs/core/perf/lmkd
- https://github.com/termux/termux-app/issues/2366 (Phantom Process Killer)
- https://github.com/termux/termux-app/issues/377 (Doze)
- https://developer.android.com/topic/performance/vitals/lmk
- https://github.com/termux/termux-app (Termux app source — `android:debuggable` confirmation)
- https://www.sitepoint.com/hardened-mobile-dev-a-termux-docker-guide-for-grapheneos/ (community discussion of test/dev workflows on Termux)